实验 11　设备管理与 SPOOLING

11.1 实验目标

(1)了解 openEuler 设备管理和 SPOOLING 原理

(2)基于 openEuler 系统分析设备管理和 SPOOLING

11.2 预备知识

11.2.1 USB 骨架(USB-Skeleton)

USB 骨架程序（usb-skeleton）是 USB 驱动程序的基础，可以看作一个最简单的 USB 设备驱动的实例。通过对它源码的学习和理解，可以使我们迅速地了解 USB 驱动架构，迅速地开发我们自己的 USB 硬件的驱动。

openEuler USB 子系统已经从仅支持 2.2.7 内核中的两种不同类型的设备（鼠标和键盘）发展到了 2.4 内核中的 20 多种不同类型的设备。 openEuler 当前支持几乎所有 USB 类设备（标准类型的设备，如键盘，鼠标，调制解调器，打印机和扬声器），以及越来越多的特定于供应商的设备（如 USB 到串行转换器，数码相机，以太网设备和 MP3 玩家）。

openEuler 上不支持的其余 USB 设备几乎是所有特定于供应商的设备。如果生产厂商在他们的产品中使用自己定义的协议，他们就需要为此设备创建特定的驱动程序。当然我们知道，有些生产厂商公开他们的 USB 协议，并帮助 openEuler 驱动程序的开发，然而有些生产厂商却根本不公开他们的 USB 协议。因为每一个不同的协议都会产生一个新的驱动程序，所以就有了这个通用的 USB 驱动骨架程序，它是以 pci 骨架为模板的。

因为每种不同的协议都会导致创建新的驱动程序，如果你准备写一个 openEuler 驱动程序，首先要熟悉 USB 协议规范。这里，通用的 USB 驱动程序框架以 pci-skeleton.c 文件为模板，内核源代码树的 drivers/usb/usb-skeleton.c 文件为我们提供了一个最基础的 USB 驱动程序，我们称之为 USB 骨架，通过它我们仅需要修改极少的部分，就可以完成一个 USB 设备的驱动。

Documentation/driver-api/usb/writing_usb_driver.rst 文件有详细介绍如何编写 USB 驱动程序。

11.2.2 SPOOLING 概述

在系统配备的设备数量远小于系统中并发运行进程数时，为了使并发进程能更好地共享系统设备，把独占设备转化为共享设备使用，操作系统提供了 SPOOLING 假脱机技术。

早期，为了缓和 CPU 的高速性与 I/O 设备低速性之间的矛盾，引入了脱机输入、脱机输出技术。该技术是利用专门的外围控制机，先将低速 I/O 设备上的数据传送到高速磁盘上，或者相反。这样，处理机需要输入数据时，便可直接从磁盘中读取数据。反之，处理机输出数据时，也可很快将数据先输出到磁盘上。事实上，系统引入多道程序技术之后，我们可以利用其中的一道程序，来模拟脱机输入时的外围控制机功能，把低速 I/O 设备上的数据传送到高速磁盘上。再利用另一道程序模拟脱机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上。这样，便可在主机的直接控制下，实现以前的脱机输入、输出功能。此时的外围操作与 CPU 对数据的处理同时进行，将这种在联机情况下实现的同时外围操作的技术称为 SPOOLING （Simultaneous Peripheral Operation On-Line）技术，或称为假脱机技术。

SPOOLING 系统构成：

（1）在磁盘上开辟出来的两个存储区域，称为输入井和输出井。输入井模拟脱机输入时的磁盘，收容 I/O 设备输入的数据；输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。

（2）内存中开辟的两个缓冲区，缓和 CPU 和磁盘之间速度不匹配的矛盾，称为输入缓冲区和输出缓冲区。输入缓冲区暂存输入设备传送的数据；输出缓冲区暂存从输出井传送的数据。

（3）输入进程或输出进程。输入进程模拟脱机输入时的外围控制机，将用户要求的数据从输入设备传送到输入缓冲区，再存放到输入井。当 CPU 需要输入设备时，直接从输入井读入内存。输出进程模拟脱机输出时的外围控制机，把用户要求输入的数据从内存传送并存放到输出井，待输出设备空闲时，再将输出井中的数据经输出缓冲区输出至输出设备上。

（4）井管理程序：控制作业与磁盘井之间信息的交换。

11.3 实验内容

11.3.1 USB 探测驱动模拟

(1)openEuler USB 基础

openEuler USB 驱动程序需要做的第一件事情就是在 openEuler USB 子系统里注册，并提供一些相关信息，例如：这个驱动程序支持那种设备，当被支持的设备从系统插入或拔出时，会有哪些动作。所有这些信息都传送到 USB 子系统中，在 USB 骨架驱动程序中是这样来表示的：

```
static struct usb_driver skel_driver = {
        .name =          "skeleton",                //名字
        .probe =         skel_probe,                //探测函数
        .disconnect =    skel_disconnect,           //抽调函数
        .suspend =       skel_suspend,
        .resume =        skel_resume,
        .pre_reset =     skel_pre_reset,
        .post_reset =    skel_post_reset,
        .id_table =      skel_table,                //支持设备列表
        .supports_autosuspend = 1,
};
```

变量 name 是一个字符串，它对驱动程序进行描述。probe 和 disconnect 是函数指针，当插入/拔出的 USB 设备与在 id_table 中变量信息匹配时，此函数被调用。

(2)USB 驱动的注册和注销

①注册 USB 驱动

通常在驱动程序的 init 函数中通过调用 usb_register 来注册 USB 驱动程序，USB 驱动程序在注册时会发送一个命令给 usb_register，通常在驱动程序的初始化函数里。

```
static int __init usb_skel_init(void)
{
        int result;

        /* register this driver with the USB subsystem */
        result = usb_register(&skel_driver);
        if (result < 0) {
            err("usb_register failed for the "__FILE__ "driver."
            "Error number %d", result);
            return -1;
```

```
        }

        return 0;
}
module_init(usb_skel_init);
```

②注销 USB 驱动

当要从系统卸载驱动程序时，它需要向 USB 子系统注销自身。即需要 usb_deregister() 函数处理：

```
static void __exit usb_skel_exit(void)
{
        /* deregister this driver with the USB subsystem */
        usb_deregister(&skel_driver);          //注销设备
}
module_exit(usb_skel_exit);
```

当 USB 设备插入时，为了使 linux-hotplug（openEuler 中 PCI、USB 等设备热插拔支持）系统自动装载驱动程序，你需要创建一个 MODULE_DEVICE_TABLE。代码如下（这个模块仅支持某一特定设备），以下代码告诉热插拔脚本，该模块支持具有特定供应商和产品 ID 的单个设备：

```
/* Define these values to match your devices */
#define USB_DETECT_VENDOR_ID       0xfff0
#define USB_DETECT_PRODUCT_ID      0xfff0

/* table of devices that work with this driver */
static struct usb_device_id skel_table [] = {                    //在系统中注册设备
{ USB_DEVICE(USB_SKEL_VENDOR_ID, USB_SKEL_PRODUCT_ID) },          //生产商号和设备号
{ }                                    /* Terminating entry */
};
MODULE_DEVICE_TABLE (usb, skel_table);
```

其中，USB_DETECT_VENDOR_ID 与 USB_DETECT_PRODUCT_ID 对应 lsusb 输出列表中的 ID 信息。

(3)设备操作接口说明

①探测函数 probe

USB_DEVICE 宏利用厂商 ID 和产品 ID 为我们提供了一个设备的唯一标识。当系统插入一个 ID 匹配的 USB 设备到 USB 总线时，驱动会在 USB core 中注册。驱动程序中 probe 函数也就会被调用。usb_device 结构指针、接口号和接口 ID 都会被传递到函数中。

```
// 参见 drivers/usb/usb-skeleton.c
static int skel_probe(struct usb_interface *interface, const struct usb_device_id *id)
```

驱动程序需要确认插入的设备是否可以被接受，如果是，则返回 0；如果不是，或者初始化期间发生任何错误，则从探测函数返回错误代码（例如-ENOMEM 或-ENODEV）。

在骨架驱动程序中，我们确定将哪些端点标记为大进大出。我们创建了缓冲区来保存将要从设备发送和接收的数据，并且已初始化了将数据写入设备的 USB urb（URB，USB Request Block，USB 请求块）。

相反，将设备从 USB 总线中拔出时，设备指针会调用 disconnect 函数。驱动程序需要清除此时已分配的所有私有数据，并关闭 USB 系统中所有待定的 urb。

②抽调函数 disconnect

当用户程序释放它用来与设备通信的文件句柄时，将调用驱动程序中的 skel_disconnect 函数。

```
// 参见 drivers/usb/usb-skeleton.c
static void skel_disconnect(struct usb_interface *interface)
```

③open 函数

现在，skeleton 驱动就已经和设备绑定上了，任何用户态程序要操作此设备都可以通过 file_operations 结构所定义的函数进行了。当程序尝试为 I/O 打开设备时，首先被调用的函数是 open()。

```
/ 参见 drivers/usb/usb-skeleton.c
static int skel_open(struct inode *inode, struct file *file)
```

当 open 完设备后，skel 的 read、write 函数就可以收、发数据了，他们是完成驱动对读写等操作的响应。

④write 函数

```
// 参见 drivers/usb/usb-skeleton.c
static ssize_t skel_write(struct file *file, const char *user_buffer, size_t count, loff_t*ppos)
```

参数说明：

struct file *file	待写入的文件指针
const char *user_buffer	指向用户想要发送到设备的数据的指针
size_t count	发送数据的大小

loff_t *ppos　　在当前文件中进行写入操作的位置/偏移量

该函数根据已创建的写入 urb 的大小来确定可以发送给设备的数据量（此大小取决于设备具有的批量输出端点的大小）。 然后，它将数据从用户空间复制到内核空间，将 urb 指向数据，并将 urb 提交给 USB 子系统。

⑤read 函数

```
// 参见 drivers/usb/usb-skeleton.c
static ssize_t skel_read(struct file *file, char *buffer, size_t count, loff_t *ppos)
```

参数说明：

struct file *file　　待写入的文件指针

const char *user_buffer　　指向用户想要接收的设备的数据的指针

size_t count　　接收数据的大小

loff_t *ppos　　在当前文件中进行读取操作的位置/偏移量

read 函数与 write 函数稍有不同在于：程序并没有用 urb 将数据从设备传送到驱动程序，而是我们用 usb_bulk_msg 函数代替，这个函数能够在不需要创建 urbs 和操作 urb 函数的情况下，来发送数据给设备，或者从设备来接收数据。我们调用 usb_bulk_msg 函数，并为它提供一个缓冲区，该缓冲区用于放置从设备接收到的所有数据以及超时值。 如果超时期限未收到设备的任何数据，则该功能将失败并返回错误消息。

⑥delete 函数

由于USB 设备可以在任何时间点从系统中取走,即使程序目前正在访问它。USB 驱动程序必须要能够很好地处理解决此问题，它需要能够切断任何当前的读写，同时通知用户空间程序：USB 设备已不存在。此操作由 skel_delete 函数执行。

如果程序有一个打开的设备句柄，在当前结构里，我们只要把它赋值为空，就像它已经消失了。对于每一次设备读写等其它函数操作，我们都要检查 usb_device 结构是否存在。如果不存在，就表明设备已经消失，并返回一个 -ENODEV 错误给用户程序。当最终我们调用 release 函数时，在没有文件打开这个设备时，无论 usb_device 结构是否存在、它都会清空 skel_disconnect 函数所作工作。

(4)用 lsusb 命令查看 U 盘的信息

在 Linux 中我们可以使用 lsusb 来列出 USB 设备和它的属性，lsusb 会显示驱动和内部连接到你系统的设备。直接在控制台输入 lsusb 即可。

用法： lsusb [options]...

功能：查看/列出 USB 设备

参数说明：（可通过 lsusb --help 查看详细使用说明）

-v, --verbose	增加详细程度（显示描述符），列出所有 USB 的详细
-s [[bus]:][devnum]	仅显示具有指定设备和/或总线号的设备（十进制）
-d vendor:[product]	仅显示具有指定供应商和产品 ID 号（十六进制）的设备
-D device	选择 lsusb 将检查的设备
-t, --tree	将物理 USB 设备层次结构转储为树
-V, --version	显示程序版本
-h, --help	显示用法和帮助

如果无法运行 lsusb，使用以下命令安装：

```
dnf install -y usbutils
```

运行 lsusb 的输出结果如下：

```
[root@localhost ~]# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 005: ID 0e0f:0008 VMware, Inc. VMware Virtual USB Mouse
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

输出内容说明：

①Bus 001

表示第 1 个 USB 主控制器（机器上总共有 1 个 USB 主控制器 -- 可以通过命令 lspci | grep USB 查看，dnf install -y pciutils 可安装 lspci 命令）

```
[root@localhost ~]# lspci | grep USB
02:00.0 USB controller: VMware USB1.1 UHCI Controller
02:03.0 USB controller: VMware USB2 EHCI Controller
```

②Device 001

表示系统给 USB 设备分配的设备号(devnum)，同时也可以看到该设备是插入到了第一个 USB 主控制器

③ID 1d6b:0002

表示 USB 设备的 ID（这个 ID 由芯片制造商设置，可以唯一表示该设备）

1d6b　　USB_DETECT_VENDOR_ID

0002　　USB_DETECT_PRODUCT_ID

(5)实验基本步骤

参考内核源码中的 drivers/usb/usb-skeleton.c 文件，编写一个 USB 探测驱动程序，能够实现基本功能：（1）在插入 U 盘时能够探测到；（2）在拔出 U 盘时能够探测到。加载、卸载模块并查看模块打印信息。

①打开 FinalShell 窗口，创建一个目录，命名为 usb_demo1, cd 到该目录下，创建一个 C 文件，命名为 usb_detect，并打开该文件：

```
[root@localhost ~]# mkdir usb_demo1
[root@localhost ~]# cd usb_demo1
[root@localhost usb_demo1]# touch usb_detect.c
[root@localhost usb_demo1]# vi    usb_detect.c
```

②按“i”键进入 Insert 模式，在 usb_detect.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <linux/kernel.h>
#include <linux/errno.h>
#include <linux/slab.h>
#include <linux/module.h>
#include <linux/kref.h>
#include <linux/uaccess.h>
#include <linux/usb.h>
#include <linux/mutex.h>
/* Define these values to match your devices */
#define USB_DETECT_VENDOR_ID 0x090c
#define USB_DETECT_PRODUCT_ID       0x2000

/* table of devices that work with this driver */
static const struct usb_device_id usbdetect_table[] = {
{ USB_DEVICE(USB_DETECT_VENDOR_ID, USB_DETECT_PRODUCT_ID) },
      { }                           /* Terminating entry */
};
MODULE_DEVICE_TABLE(usb, usbdetect_table);

/* Get a minor range for your devices from the usb maintainer */
#define USB_DETECT_MINOR_BASE      192
#define WRITES_IN_FLIGHT     8
/* arbitrarily chosen */
/* Structure to hold all of our device specific stuff */
struct usb_detect {
```

```
        struct usb_device    *udev;                 /* the usb device for this device */
        struct usb_interface *interface;            /* the interface for this device */
        struct semaphore     limit_sem;             /* limiting the number of writes in progress */
        struct usb_anchor    submitted;             /* in case we need to retract our submissions */
        struct urb       *bulk_in_urb;      /* the urb to read data with */
unsigned char                *bulk_in_buffer; /* the buffer to receive data */
        size_t               bulk_in_size;      /* the size of the receive buffer */
        size_t               bulk_in_filled;    /* number of bytes in the buffer */
        size_t               bulk_in_copied;          /* already copied to user space */
        __u8                 bulk_in_endpointAddr;  /* the address of the bulk in endpoint */
        __u8                 bulk_out_endpointAddr; /* the address of the bulk out endpoint */
        int              errors;                /* the last request tanked */
        bool             ongoing_read;      /* a read is going on */
        spinlock_t           err_lock;      /* lock for errors */
        struct kref          kref;
        struct mutex         io_mutex;              /* synchronize I/O with disconnect */
        unsigned long        disconnected:1;
        wait_queue_head_t bulk_in_wait;     /* to wait for an ongoing read */
};
#define to_detect_dev(d) container_of(d, struct usb_detect, kref)

static struct usb_driver usbdetect_driver;

static void usbdetect_delete(struct kref *kref)
{
        struct usb_detect *dev = to_detect_dev(kref);

        usb_free_urb(dev->bulk_in_urb);
        usb_put_intf(dev->interface);
        usb_put_dev(dev->udev);
        kfree(dev->bulk_in_buffer);
        kfree(dev);
}

static const struct file_operations usbdetect_fops = {};

/*
  * usb class driver info in order to get a minor number from the usb core,
  * and to have the device registered with the driver core
  */
static struct usb_class_driver usbdetect_class = {
        .name =         "usbdetect%d",
        .fops =         &usbdetect_fops,
        .minor_base = USB_DETECT_MINOR_BASE,
```

```
};
static int usbdetect_probe(struct usb_interface *interface,
                           const struct usb_device_id *id)
{
        struct usb_detect *dev;
        struct usb_endpoint_descriptor *bulk_in, *bulk_out;
        int retval;

        /* allocate memory for our device state and initialize it */
        dev = kzalloc(sizeof(*dev), GFP_KERNEL);
        if (!dev)
                return -ENOMEM;

        kref_init(&dev->kref);
        sema_init(&dev->limit_sem, WRITES_IN_FLIGHT);
        mutex_init(&dev->io_mutex);
        spin_lock_init(&dev->err_lock);
        init_usb_anchor(&dev->submitted);
        init_waitqueue_head(&dev->bulk_in_wait);

        dev->udev = usb_get_dev(interface_to_usbdev(interface));
        dev->interface = usb_get_intf(interface);

        /* set up the endpoint information */
        /* use only the first bulk-in and bulk-out endpoints */
        retval = usb_find_common_endpoints(interface->cur_altsetting,
                        &bulk_in, &bulk_out, NULL, NULL);
        if (retval) {
                dev_err(&interface->dev,
                        "Could not find both bulk-in and bulk-out endpoints\n");
                goto error;
        }
        dev->bulk_in_size = usb_endpoint_maxp(bulk_in);
        dev->bulk_in_endpointAddr = bulk_in->bEndpointAddress;
        dev->bulk_in_buffer = kmalloc(dev->bulk_in_size, GFP_KERNEL);
        if (!dev->bulk_in_buffer) {
                retval = -ENOMEM;
                goto error;
        }
        dev->bulk_in_urb = usb_alloc_urb(0, GFP_KERNEL);
        if (!dev->bulk_in_urb) {
                retval = -ENOMEM;
                goto error;
        }
```

```
	dev->bulk_out_endpointAddr = bulk_out->bEndpointAddress;

	/* save our data pointer in this interface device */
	usb_set_intfdata(interface, dev);

	/* we can register the device now, as it is ready */
	retval = usb_register_dev(interface, &usbdetect_class);
	if (retval) {
		/* something prevented us from registering this driver */
		dev_err(&interface->dev,
			"Not able to get a minor for this device.\n");
		usb_set_intfdata(interface, NULL);
		goto error;
	}

	/* let the user know what node this device is now attached to */
	dev_info(&interface->dev,
		 "USB detect device now attached to USBdetect-%d",
		 interface->minor);
	return 0;

error:
	/* this frees allocated memory */
	kref_put(&dev->kref, usbdetect_delete);

	return retval;
}

static void usbdetect_disconnect(struct usb_interface *interface)
{
	struct usb_detect *dev;
	int minor = interface->minor;
	dev = usb_get_intfdata(interface);
	usb_set_intfdata(interface, NULL);

	/* give back our minor */
	usb_deregister_dev(interface, &usbdetect_class);

	/* prevent more I/O from starting */
	mutex_lock(&dev->io_mutex);
	dev->disconnected = 1;
	mutex_unlock(&dev->io_mutex);
	usb_kill_anchored_urbs(&dev->submitted);
```

```
	/* decrement our usage count */
	kref_put(&dev->kref, usbdetect_delete);

	dev_info(&interface->dev, "USB detect #%d now disconnected", minor);
}

static struct usb_driver usbdetect_driver = {
	.name =		"usbdetect",
	.probe =	usbdetect_probe,
	.disconnect =	usbdetect_disconnect,
	.id_table =	usbdetect_table,
	.supports_autosuspend = 1,
};

MODULE_LICENSE("GPL v2");

static int __init usb_detect_init(void)
{
	int result;
	printk("Start usb_detect module...");
	/* register this driver with the USB subsystem */
	result = usb_register(&usbdetect_driver);
	if (result < 0) {
		printk("usb_register failed.""Error number %d", result);
		return -1;
	}
	return 0;
}

static void __exit usb_detect_exit(void)
{
	printk("Exit usb_detect module...");
	/* deregister this driver with the USB subsystem */
	usb_deregister(&usbdetect_driver);
}

module_init(usb_detect_init);
module_exit(usb_detect_exit);
```

③编写 Makefile 文件，按“i”键进入 Insert 模式，在文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
ifneq ($(KERNELRELEASE),)
	obj-m := usb_detect.o
else
```

```
    /*选择自己的内核路径*/
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 U 盘内核模块 ：

```
[root@localhost usb_demo1]# make
[root@localhost usb_demo1]# insmod usb_detect.ko
```

⑤在电脑的 USB 接口插入 U 盘后，查看信息可见，模块检测到 U 盘插入（如最后一行所示），并且获取到的 U 盘信息与 lsusb 显示的一致：

```
[root@localhost usb_demo1]# dmesg | tail -n 20
[39726.186935] audit: type=1106 audit(1661106617.220:240553): pid=1559508 uid=0 auid=0 ses=7 msg='op=login id=0 exe="/usr/sbin/sshd" hostname=? addr=192.168.52.1 terminal=ssh res=success'
[39726.211310] audit: type=1105 audit(1661106617.246:240554): pid=1559508 uid=0 auid=0 ses=7 msg='op=login id=0 exe="/usr/sbin/sshd" hostname=? addr=192.168.52.1 terminal=ssh res=success'
[39726.213704] audit: type=2404 audit(1661106617.249:240555): pid=1559508 uid=0 auid=0 ses=7 msg='op=destroy kind=server fp=SHA256:c7:e5:04:dd:6f:68:8e:93:ce:b9:76:a7:9f:a7:0f:69:e2:a0:78:cf:a2:d4:f8:db:9d:65:46:7e:02:e5:32:35 direction=? spid=3509254 suid=0 exe="/usr/sbin/sshd" hostname=? addr=? terminal=? res=success'
[39726.679197] scsi 3:0:0:0: Direct-Access     hp    v206w     1100 PQ: 0 ANSI: 4
```

⑥从电脑中拔出 U 盘，查看打印信息可见：USB 设备已断开连接（最后一行所示）：

```
[root@localhost usb_demo1]# dmesg | tail -n 20
[39697.742003] audit: type=1106 audit(1661106586.403:240394): pid=1559508 uid=0 auid=0 ses=7 msg='op=login id=0 exe="/usr/sbin/sshd" hostname=? addr=192.168.52.1 terminal=ssh res=success'
[39697.757437] audit: type=1105 audit(1661106586.420:240395): pid=1559508 uid=0 auid=0 ses=7 msg='op=login id=0 exe="/usr/sbin/sshd" hostname=? addr=192.168.52.1 terminal=ssh res=success'
[39698.316133] usb 1-1: USB disconnect, device number 21
```

⑦可以用 lsusb 命令查看 U 盘插入和拔出的情况（输出信息的第一行）：

```
[root@localhost usb_demo1]# lsusb
Bus 001 Device 022: ID 090c:2000 Silicon Motion, Inc. - Taiwan (formerly Feiya Technology Corp.) v206w
```

```
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 005: ID 0e0f:0008 VMware, Inc. VMware Virtual USB Mouse
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
[root@localhost usb_demo1]# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 005: ID 0e0f:0008 VMware, Inc. VMware Virtual USB Mouse
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

⑧卸载模块：

```
[root@localhost usb_demo1]# rmmod usb_detect.ko
```

11.3.2 内核文件编写模拟

(1)内核文件读写相关知识

有时候需要在 Linux kernel 中读写文件数据，如调试程序的时候，或者内核与用户空间交换数据的时候。在 kernel 中操作文件没有标准库可用，需要利用 kernel 的一些函数，这些函数主要有：

filp_open()

filp_close()

kernel_read()

kernel_write()

这些函数在 <linux/fs.h> 头文件中声明。内核文件读写接口如下：

①打开文件

函数原型：struct file* filp_open(const char* filename, int open_mode, int mode);

功能：在 kernel 中打开指定文件。

返回值：该函数返回 struct file* 结构指针，供后续函数操作使用；该返回值用 IS__ERR（）来检验其有效性。

参数说明：

filename：表明要打开或创建文件的名称（包括路径部分）。注意：在内核中打开文件时需要注意打开的时机，很容易出现需要打开文件的驱动很早就加载并打开文件，但需要打开的文件所在设备还没有挂载到文件系统中，而导致打开失败。

open_mode：文件的打开方式，其取值与标准库中的 open 相应参数类似。包括：O_RDONLY（只读打开）、O_WRONLY（只写打开）、O_RDWR（读写打开）、O_CREAT（文件不存在则创建）等。

mode：创建文件时使用，设置创建文件的读写权限（如 644），其它情况可以设为 0。

②读文件

函数原型：ssize_t kernel_read(struct file *file, void *buf, size_t count, loff_t *pos);

功能：kernel 中文件的读操作。

参数说明：

file：进行读取信息的目标文件，即 file_open() 函数的返回值。

buf：对应放置信息的缓冲区。

count：要读取的信息长度。

pos：表示用户在当前文件中进行读取操作的位置/偏移量，即读的位置相对于文件开头的偏移。在读取信息后，这个指针一般都会移动，移动的值为要读取信息的长度值。

③写文件

函数原型：ssize_t kernel_write(struct file *file, const void *buf, size_t count, loff_t *pos);

功能：kernel 中文件的写操作。

参数说明：

file：进行信息写入的目标文件，即 file_open() 函数的返回值。

buf：要写入文件的信息缓冲区。

count：要写入信息的长度。

pos：表示用户在当前文件中进行写入操作的位置/偏移量。即写的位置相对于文件开头的偏移。

④关闭文件

函数原型：int filp_close(struct file*filp, fl_owner_t id);

功能：关闭指定文件。

参数说明：

filp：待关闭的目标文件的文件指针。

id：一般传递 NULL 值，也可用 current->files 作为实参。

(2)IOZONE 工具

IOZONE 是一个文件系统的 benchmark 工具，可以测试不同的操作系统中文件系统的读写性能。测试的时候请注意，设置的测试文件的大小一定要大过你的内存（最佳为内存的两倍大小），不然 linux 会给你的读写的内容进行缓存，会使数值非常不真实。

安装使用如下：

```
# wget http://www.iozone.org/src/current/iozone3_489.tar
# tar -xvf iozone3_489.tar
# cd iozone3_489
# ll
total 8.0K
drwxr-xr-x 2 root root 4.0K Jul    6 03:13 docs
drwxr-xr-x 3 root root 4.0K Jul    6 03:13 src
# cd src/
# ll
total 4.0K
drwxr-xr-x 2 root root 4.0K Jul    6 03:24 current
# cd current/
# make linux-AMD64
# ll -rt       列表中随后这三个就是可用的执行文件
-rwxr-xr-x 1 root root 353K Jul    6 03:24 iozone
-rwxr-xr-x 1 root root    77K Jul    6 03:24 fileop
-rwxr-xr-x 1 root root    72K Jul    6 03:24 pit_server
#  查看具体用法
# ./fileop -h
# ./pit_server -h
# ./iozone -h
```

使用 IOZONE 可以在多线程、多 CPU，并指定 CPU Cache 空间大小以及同步或异步 I/O 读写模式的情况下进行测试文件操作性能；

命令格式：iozone [options]　这里用 ./iozone [options]

常用参数：

-a	全自动模式测试。测试记录块大小从 4k 到 16M；耗时长，最好用-i 指定测试范围。
-b	指定输出到指定文件上。
-C	显示每个节点的吞吐量。
-c	测试包括文件的关闭时间。
-f filename	指定测试文件的名字，完成后会自动删除（这个文件必须指定

	在你要测试的那个硬盘中）。
-g	在自动模式下设置文件最大值（指定最大测试文件大小）。 可以使用 k 、m 、g 分别表示 kb，mb，gb。如：-g 12k
-n	指定最小测试文件大小。
-i	指定运行于哪种模式测试。可以进行多个模式测试，例如：-i 0 -i 1 -i 5： 0=write/rewrite 1=read/re-read 2=random read/random write 3=backwards read 4=re-write-record 5=stride-read 6=fwirte/re-fwrite 7=fread/re-fread 8=random mix 9=pwrite/re-pwrite 10=pread/re-pread 11=pwritev/re-pwritev 12=preadv/re-preadv
-q	在自动模式下设置记录块的最大值，可以使用 k(kb)，m(mb)，g(gb)。 使用-y 可以设置最小值。（例如： -q 64k 包括了 4K,8K,16K,32K,64K）。
-r	指定一次写入/读出的块大小（与-q 有别，-r 64k 只进行 64k 的测试）。
-R	产生 execl 格式的输出日志。
-s	设置测试文件大小。
-y	设置记录块最小值。
-z	同 -a 一起使用，进行全部测试。

实例：

#./iozone -a -n 512m -g 16g -i 0 -i 1 -i 5 -Rb ./iozone.xls

说明：进行全面测试，最小测试文件为 512M，直到测试到 16g，测试 read、write，生成 xls 文件。要求-g 参数大于内存的两倍。可使用 free -m 查看内存大小。

(3)实验步骤

编写内核模块测试硬盘的读写速率，并用 IOZONE 工具测试用户态下硬盘的读写速率。

①先编写内核模块测试硬盘的写速率。打开 FinalShell 窗口，创建一个目录，命名为 usb_demo2, cd 到该目录下，创建一个 C 文件，命名为 write_to_disk，并打开该文件：

```
[root@localhost ~]# mkdir usb_demo2
[root@localhost ~]# cd usb_demo2
[root@localhost usb_demo2]# touch write_to_disk.c
[root@localhost usb_demo2]# vi    write_to_disk.c
```

②按“i”键进入 Insert 模式，在 write_to_disk.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/rtc.h>

#define buf_size 1024
#define write_times 524288

MODULE_LICENSE("GPL");

struct timeval tv;

static int __init write_disk_init(void)
{
	struct file *fp_write;
	char buf[buf_size];
	int i;
	int write_start_time;
	int write_start_time_u;
	int write_end_time;
	int write_end_time_u;
	int write_time;
	loff_t pos;
	printk("Start write_to_disk module...\n");
	for(i = 0; i < buf_size; i++)
	{
		buf[i] = i + '0';
	}
	fp_write = filp_open("/home/tmp_file", O_RDWR | O_CREAT,0644);
	if (IS_ERR(fp_write)) {
		printk("Failed to open file...\n");
		return -1;
	}
	pos = 0;
	do_gettimeofday(&tv);
	write_start_time = (int)tv.tv_sec;
	write_start_time_u = (int)tv.tv_usec;
```

```
    for(i = 0; i < write_times; i++) {
        kernel_write(fp_write, buf, buf_size, &pos);
    }
    do_gettimeofday(&tv);
    write_end_time = (int)tv.tv_sec;
    write_end_time_u = (int)tv.tv_usec;
    filp_close(fp_write, NULL);
    write_time = (write_end_time - write_start_time)  * 1000000 + (write_end_time_u - write_start_time_u);
    printk(KERN_ALERT "Writing to file costs %d us\n", write_time);
    printk("Writing speed is %d M/s\n", buf_size * write_times / write_time);
    return 0;
}

static void __exit write_disk_exit(void)
{
    printk("Exit write_to_disk module...\n");
}

module_i nit(write_disk_init);
module_exit(write_disk_exit);
```

③编写 Makefile 文件，按“i”键进入 Insert 模式，在文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
ifneq ($(KERNELRELEASE),)
    obj-m := write_to_disk.o
else
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 U 盘内核模块：

```
[root@localhost usb_demo2]# make
make -C /root/kernel-4.19.90-2108.8.0 M=/root/usb_demo2 modules
make[1]: 进入目录“/root/kernel-4.19.90-2108.8.0”
  CC [M]  /root/usb_demo2/write_to_disk.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC          /root/usb_demo2/write_to_disk.mod.o
  LD [M]  /root/usb_demo2/write_to_disk.ko
```

```
make[1]: 离开目录“/root/kernel-4.19.90-2108.8.0”
[root@localhost usb_demo2]# insmod write_to_disk.ko
```

⑤测试写速率：

```
[root@localhost usb_demo2]# dmesg | tail -n 3
[45765.184156] Writing to file cost 3512146 us
[45765.184213]Writing speed is 152 M/s
```

⑥卸载模块：

```
[root@localhost usb_demo2]# rmmod write_to_disk.ko
```

⑦编写内核模块测试硬盘的读速率。和上述步骤一样，创建目录 usb_demo3,cd 到该目录下，创建 read_from_disk.c 文件和 Makefile 文件，在文件中分别写入如下内容，编译加载模块并测试即可：

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/rtc.h>

#define buf_size 1024
#define read_times 524288

MODULE_LICENSE("GPL");

struct timeval tv;

static int __init read_disk_init(void)
{
    struct file *fp_read;
    char buf[buf_size];
    int i;
    int read_start_time;
    int read_start_time_u;
    int read_end_time;
    int read_end_time_u;
    int read_time;
    loff_t pos;
    printk("Start read_from_disk module...\n");
    fp_read = filp_open("/home/tmp_file", O_RDONLY, 0);
    if (IS_ERR(fp_read)) {
        printk("Failed to open file...\n");
        return -1;
    }

    do_gettimeofday(&tv);
    read_start_time = (int)tv.tv_sec;
```

```
    read_start_time_u = (int)tv.tv_usec;
    pos = 0;
    for(i = 0; i < read_times; i++) {
        kernel_read(fp_read, buf, buf_size, &pos);
    }
    do_gettimeofday(&tv);
    read_end_time = (int)tv.tv_sec;
    read_end_time_u = (int)tv.tv_usec;
    filp_close(fp_read, NULL);
    read_time = (read_end_time - read_start_time) * 1000000 + (read_end_time_u - read_start_time_u);
    printk(KERN_ALERT "Read file costs %d us\n", read_time);
    printk("Reading speed is %d M/s\n", buf_size * read_times / read_time);
    return 0;
}

static void __exit read_disk_exit(void)
{
    printk("Exit read_from_disk module...\n");
}

module_init(read_disk_init);
module_exit(read_disk_exit);
```

```
ifneq ($(KERNELRELEASE),)
    obj-m := read_from_disk.o
else
    KERNELDIR ?= /root/kernel-4.19.90-2108.8.0
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY:clean
clean:
```

```
[root@localhost usb_demo3]# dmesg | tail -n 3
[869427.516557]Read file costs 1191551 us
[869427.516591]Reading speed is 450 M/s
```

⑧IOZONE 工具测试，先查看确认当前环境：

```
[root@localhost current]# free -m
              total        used        free      shared  buff/cache   available
Mem:           3406         201        2363         169         841        2685
Swap:          4047           0        4047
```

```
[root@localhost current]# pwd
/root/iozone3_489/src/current
```

⑨执行测试：

```
[root@localhost current]# ./iozone -Raz -n 512m -g 8g -r 1k -i 0 -i 1 -b /home/iozone.xls
iozone test complete.
Excel output is below:

"Writer report"
        "1"
"524288"    413498
"1048576"    511930
"2097152"    546979
"4194304"    724829
"8388608"    848518

"Re-writer report"
        "1"
"524288"    2025318
"1048576"    1530508
"2097152"    667309
"4194304"    33116
"8388608"    24207
"Reader report"
        "1"
"524288"    3424326
"1048576"    3867584
"2097152"    4300782
"4194304"    1380041
"8388608"    1288000

"Re-Reader report"
        "1"
"524288"    4064596
"1048576"    4253928
"2097152"    3837908
"4194304"    1400134
"8388608"    1251193
```

上述结果中是关于 read、write 等的测试结果，左侧第一列是测试文件的大小（kbytes）；最上面一行的 1 是记录块大小，即 1Kb；中间数据是测试的传输速度。举例说明，比如“Write report”中第一行数据的“413498”，意思是测试文件大小为 512M，以大小为 1K（1024byte）的记录块进行传输，它的传输速度为

413498/s（kb/s），即约 403.81M/s。同理可得到读的速率。

11.3.3 SPOOLING 假脱机模拟

SPOOLING 进程和每个用户进程都有相应的进程控制块表示。进程的状态有以下几种：可执行态 0 表示进程正在执行或等待调度；

等待态 1 表示输出请求块满，请求输出的用户进程等待；

等待态 2 表示输出井空，SPOOLING 输出进程等待；

等待态 3 表示输出井满，请求输出的用户进程等待；

完成态　　表示执行完成。

进程状态转换条件是：

（1） 各进程初始都为可执行态；

（2） 服务程序在将信息送输出井时，发现输出井已满，将调用进程置为等待态 1；

（3） SPOOLING 进程执行输出时，若输出井为空，则置为等待态 2；

（4） SPOOLING 进程完成一个输出请求块的信息输出后，应立即释放该请求块所占的输出井空间，并将在等待输出井（若有）的用户进程置为可执行态；

（5） 服务程序将输出信息送入输出井并形成输出请求块后，发现 SPOOLING 进程处于等待态 2，应把它置为可执行态；

（6） 用户进程申请输出请求块时，没有可用的请求块，置为等待态 3；

（7） 用户进程完成全部输出任务后，置为完成态。

进程调度采用随机算法。两个请求输出的用户进程的调度概率各为 45%，SPOOLING 输出进程为 10%，这由随机数发生器产生的随机数来模拟确定。

输出进程的输出信息可假定为一串随机数，当遇到数为 0 时，表示该次输出结束。

输出请求块可以定义为一个结构数组，输出井 BUFFER 可定义为一个整型数组。两者都是循环使用。为此，每种数据都有两个指针：一个是使用（放数）指针，一个是释放（取数）指针。

程序中应显示和打印用户进程送入输出井的信息和 SPOOLING 进程输出的信息，检查两者是否一致。

实验步骤

设计一个 SPOOLING 假脱机输出模拟程序。

①打开 FinalShell 窗口，创建一个 C 文件，命名为 spooling，并打开 spooling.c 文件：

```
[root@localhost ~]# touch spooling.c
[root@localhost ~]# vi spooling.c
```

②按“i”键进入 Insert 模式，在 spooling.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
struct pcb {
    int id;
    int status;
    int firstaddr;
    int length;
    int outbufword;
}*PCB[3];
FILE *f;
struct req { //输出请求块
    int reqname;//请求输出的用户进程名
    int length;//要输出信息的长度
    int addr;//输出信息在输出井中的位置
}reqblock[10];
int buffer[2][100];//为两个用户进程设置两个输出井，每个可存放 100 个信息
int c3=10;
int l1=1, l2=1;
int head=0, tail=0;
int t1, t2;
void request(int i) {
    int j, length=0, m;
    struct req *run;
    if( (tail-head) == 10 ) {
        PCB[i-1]->status =1;
        return;
    }
    if(i==1) t1--;
    else t2--;
    run = &reqblock[tail%10];
    run->reqname = i;
    run->length = 0;
    if(tail == 0) run->addr = 0;
    else { int index = (tail-1)%10;
```

```
        run->addr = reqblock[index].addr + reqblock[index].length;
}
  for(m=0; m<100; m++)
  { if(buffer[i-1][m] == 0)
  { run->addr = m;
  break;
}
  }
  printf("process %d: ",i);
  while(1)
  { j = rand()%10;
  if(j == 0)
  { run->length = length;
  break;
  }
  buffer[i-1][run->addr + length] = j;
  printf("%d ",j);
  length++;
  }
  printf("\n");
  PCB[i-1]->length += length;
  length = 0;
  if(PCB[2]->status == 2) PCB[2]->status = 0;
  tail++;
}
void spooling()
{ int i,j;
  struct req *run;
  run = &reqblock[head%10];
  printf("PID %d: ",run->reqname);
  fprintf(f,"PID %d:",run->reqname);
  for(i=0; i<run->length; i++)
  { printf("%d ",buffer[run->reqname - 1][run->addr + i]);
  fprintf(f,"%d ",buffer[run->reqname - 1][run->addr + i]);
  }
  printf("\n");
  fprintf(f,"\n");
  head++;
  for(j=0; j<2; j++)
  if(PCB[j]->status == 1) PCB[j]->status = 0;
}
int main()
{ int i,n;
  f = fopen("result.txt","w");
```

```
  for(i=0; i<2; i++)
  for(n=0; n<100; n++)
  buffer[i][n] = 0;
  for(i=0; i<3; i++)
  { struct pcb *temppcb;
    temppcb = (struct pcb *)malloc(sizeof(struct pcb));
    temppcb->id = i;
    temppcb->status = 0;
    temppcb->firstaddr = 0;
    temppcb->length = 0;
    temppcb->outbufword = 1;
    PCB[i] = temppcb;
}
  printf("How many work do process 1 want to do? ");
  fprintf(f,"How many work do process 1 want to do? ");
  scanf("%d",&t1);
  fprintf(f,"%d\n",t1);
  printf("How many work do process 2 want to do? ");
  fprintf(f,"How many work do process 2 want to do? ");
  scanf("%d",&t2);
  fprintf(f,"%d\n",t2);
  srand((unsigned)time(NULL));
  while(1)
  { i = rand()%100;
  if(i<=45)
  { if( (PCB[0]->status == 0) && (t1>0) )
  request(1);
  }
  else if(i<=90)
  { if( (PCB[1]->status == 0) && (t2>0) )
  request(2);
  }
  else if(head==tail) PCB[2]->status = 2;
  else spooling();
  if( (t1==0) && (t2==0) && (head==tail) ) break;
  }
  for(i=0; i<3; i++)
  { free(PCB[i]);
  PCB[i] = NULL;
  }
  fclose(f);
  return 0;
}
```

③编译 spooling.c 文件，生成可执行文件 spooling，并执行：

```
[root@localhost ~]# gcc spooling.c -o spooling
[root@localhost ~]# ./spooling
How many work do process 1 want to do? 2
How many work do process 2 want to do? 3
process 1: 2 6 6 1
process 2: 6 4 3 1 7 3 2 1 2 1 8 8 2 3
process 2: 5 2 9 9 6 3 2 6 6 6 2 5 7 4 3 6
PID 1: 2 6 6 1
process 2: 7 9 1 9 3 5 9 2 9 5 4 4
process 1: 5 9 4
PID 2: 6 4 3 1 7 3 2 1 2 1 8 8 2 3
PID 2: 5 2 9 9 6 3 2 6 6 6 2 5 7 4 3 6
PID 2: 7 9 1 9 3 5 9 2 9 5 4 4
PID 1: 5 9 4
```

进程的作业量可自行定义，得到不同结果。

11.4 实验讨论

1、对比用户态和内核态下测试的读写速率，并分析原因。

2、SPOOLING 假脱机技术有什么特点？

3、使用 SPOOLING 技术实现打印系统，当多个用户进程请求打印输出时，SPOOLING 系统会如何做？